VDK/TLM Creation

Rapid Training

X-2025.06

Module 2: Lab B

Implement the Behavior of an SPI Controller

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview 2
Task-1: Setup and Open the SPI project 2
Task-2: Behavior Implementation with TLMC Re-Use Objects 3
Task-3: Reuse-Object Documentation 5
Task-4: Implementing the Transmission Control (TX_PATH) 6
Task-5: The TX_PATH Internals 8
Task-6: Using The TX_PATH instance 11
Task-7: Linking TX_PATH Behavior to the Memory Map Elements 14
Task-8: Implementing the RX_PATH Object (reception control) 16
Task-9: Using the RX_PATH Instance 18
Task-10: Configuring the SPI Protocol Engine 21
What's Next 23

Lab Overview

In the previous lab Module2LabA, you've been creating the shell of a Serial Protocol Interface (SPI) controller peripheral – the task for this lab is to implement the behavior of the model. Using the library of re-use objects in the TLM Creation perspective, you can keep the amount of code you have to write and test at a minimum, and features such as continuous assignments also help to keep the model implementation concise and efficient.

Please refer to the SPI Controller Specification for interface and behavior of this training example model.

Task-1: Setup and Open the SPI project

- ***Motivation**: We had already created the shell of the SPI model in Module2Lab A. Now, we will add some additional behavior to the model implementation and add required tests to validate them.*

1. If you have completed the previous lab (Module 2 Lab A), you can continue with your existing TLM Creator project and skip reading the rest of this task and continue with *`Task-2`*.

   Otherwise, if want to start from a prepared starting point for this lab, use the following steps to create the starting point.

   i. Open a console and invoke Virtualizer studio from the Synopsys Virtual Prototyping installation.

   ```
   % source <installDir>/SLS/windows/setup.bat vi[rtualizer] (Windows)

   % source <installDir>/SLS/linux/setup.csh|sh vi[rtualizer] (Linux)
   ```

   This sourcing option picks up the Virtualizer *Elite* or *Base* license in that order, whichever is available. For more details on other options, refer to the VP documentation.

   i. Go to the `<Lab-data>/data/module_2` directory and start Virtualizer Studio.

   ```
   vs -d start_Module2LabB &
   ```

2. Close the welcome screen and import the starting point TLM Creator Project by clicking ***File*** → ***Import…*** menu option.

   i. In the ***Import*** dialog, select ***General*** → ***Existing Projects into Workspace*** to select what you want to import.

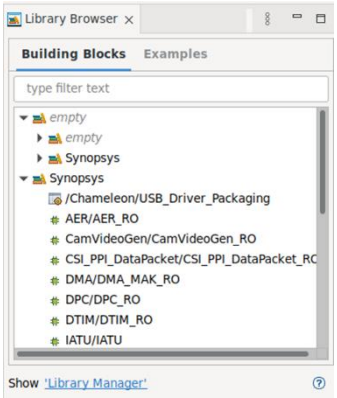
   ii. Click ***Next*** to get to the ***Import Projects*** dialog. Browse to the `SPICtrl` directory in the `start_Module2LabB` directory and click ***Finish*** (Make sure you are in the right directory – it's easy to end up in the wrong directory!).

   iii. Select the `SPICtrl.tlmc` file in the Project Explorer and click ***Generate Sources Code*** from the ***Build*** toolbar button options. Also, generate the documentation using the ***Generate Documentation*** option.

This will provide you with the starting point SPI Controller model for this lab.

Task-2: Behavior Implementation with TLMC Re-Use Objects

- ***Motivation**: This task describes how to use TLM Creation perspective to structure the behavior of a component using re-use objects. Using hierarchies and creating 'clean' interfaces between the different sub-components promotes reuse of code between models. For this, you can leverage the default object libraries added in the tool through the Library Manager.*



The SPI controller requires internal transmission and receiving FIFOs. Moreover, we will structure the internal behavior of the controller using the SCML2 behavior library part of the tool installation.

Both the FIFO and the behavioral objects are part of the `objects.tlmclib` library, which is added to the tool by default.

This library and the objects that it contains can be inspected from the ***Building Blocks*** tab of the ***Library Browser*** view, under the `${SNPS_VP_HOME}/tlmcreator/lib` category.

The Behavior section of the [SPI Controller Specification] features a block diagram. This block diagram can be implemented in the tool using the ***Instances*** tab. Both the `TX PATH` and `RX PATH` are constructed as sub-elements for the module. They will be created from generic reuse objects extended with customization behavior.

3. In the ***Instances*** tab, create a new re-use object using the following steps.

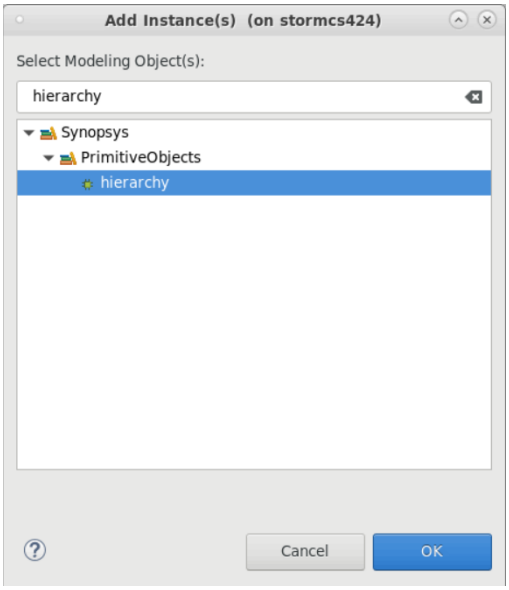
   i. Click the ***Add Object*** icon ( ) in the top-right corner.

      The Add Instance(s) dialog appears.

   ii. Type `hierarchy` in the Select Modeling Object(s) field to look for the `hierarchy` object in the Modelling object library.

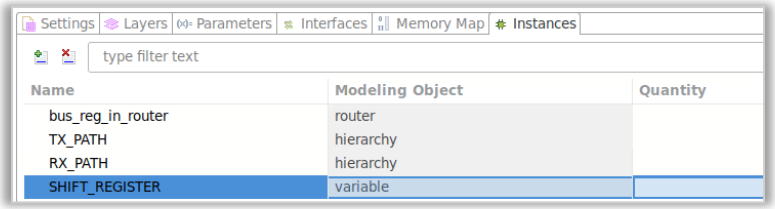
   iii. Select `hierarchy` instance under *`PrimitiveObjects`* library and click ***OK***.

   iv. Rename the object as `TX_PATH`.

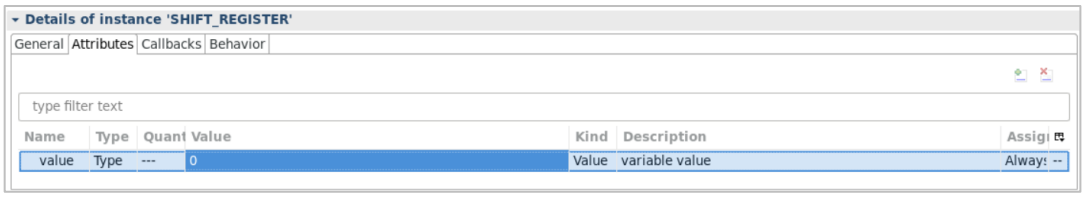


4. Similarly, add another `hierarchy` type re-use object as `RX_PATH`. Be careful: this object is not to be created as a child of the `TX_PATH` object. For that, unselect the `TX_PATH` object by using `<CTRL-left click>` before adding the new hierarchy object.

5. Add another object with ***Name*** `SHIFT_REGISTER` of `variable` type from *`PrimitiveObjects`* library.

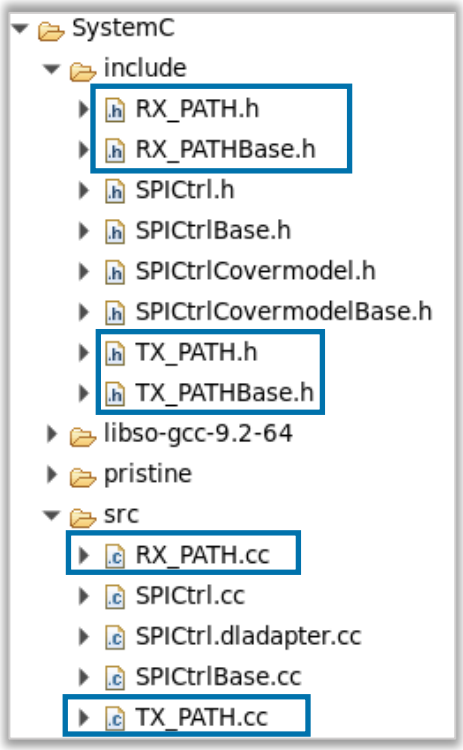


6. In the ***Details of instance 'SHIFT_REGISTER'*** panel, go to the ***General*** tab, check that the value of the ***Template Argument*** *Type* is set to `unsigned int`. Also, set the ***value*** attribute in the ***Attributes*** tab to '0'.



7. Save and build the project.

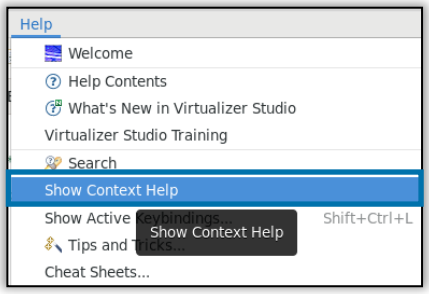
   Note: Hierarchical reuse objects are generated as separate classes with their own base class in separate header and cpp files. This leads to the project structure shown on the right-hand side.



Task-3: Reuse-Object Documentation

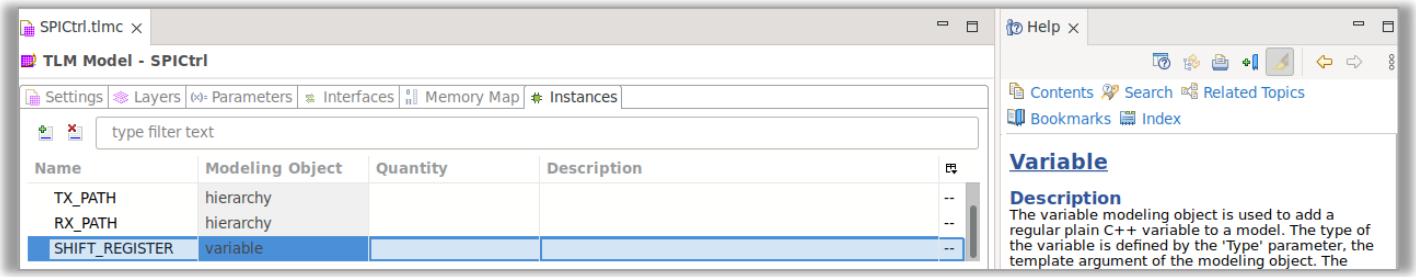
- ***Motivation**: It is important that reuse objects come with integrated documentation that can be consulted by the modeler. It should describe aspects such as "What is its main purpose?", "What are the right usage for attributes, callbacks and behaviors?" etc. The generic objects library comes with embedded documentation that can be seen from within Virtualizer Studio. This approach can be extended to custom objects as well.*

8. First thing to do it is to enable the ***Contextual Help*** panel. For that, select the ***Help->Show Contextual Help*** entry from the top menu. This will open the ***Help*** panel with contextual information.



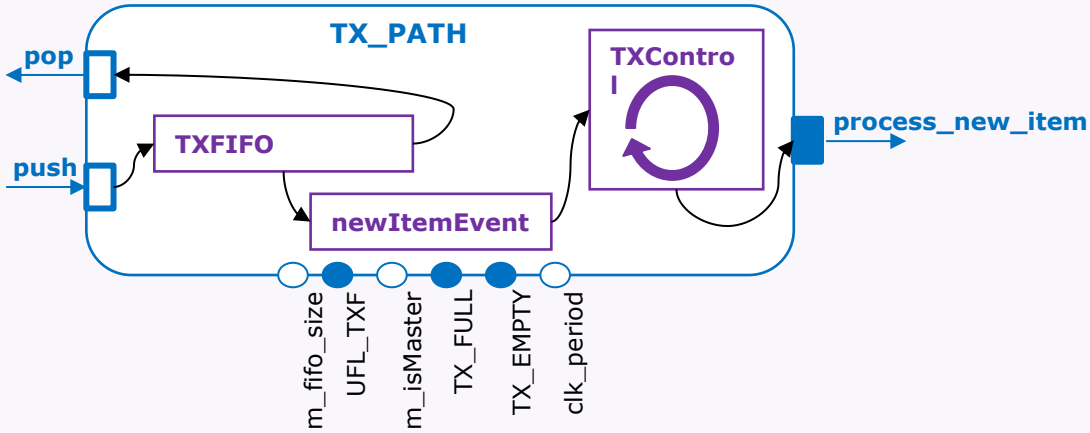
Now you can go the ***Instances*** tab and select any of the instantiated objects to populate the documentation in the ***Help*** window.

Similar context help is also available for the protocol engines of the interfaces. In this case the context help is accessible via the Attributes and Callbacks sub-tab.



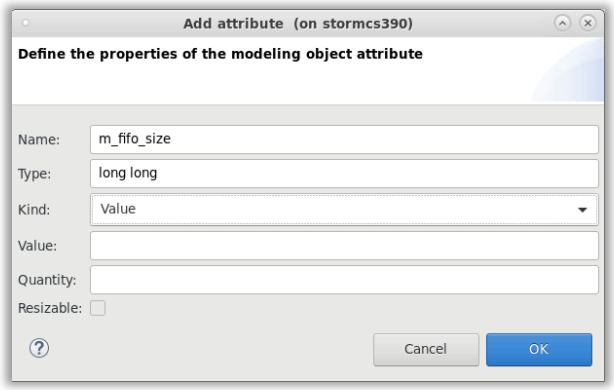
Task-4: Implementing the Transmission Control (TX_PATH)

- ***Motivation**: In this task, we will implement the functionality for the different objects in the model, starting with the `TX_PATH` i.e. the transmission control. A schematic view of this new reuse object is shown below:*



Following the reuse strategy and for optimal structuring of the design, the `TX_PATH` hierarchy is to be constructed like a re-use object. This means that the code within the `TX_PATH` object should be independent of the *SPICtrl* model. For this purpose, we need to define the attributes, callbacks and behaviors of the object as follows.

9. In the ***Attributes*** sub-tab for the `TX_PATH` object, use the ***Add new attribute*** icon ( ), or double-click into the table to open the ***Add attribute*** dialog.



Add the following attributes (one by one - no ***Quantity*** to be set for any attribute):

i. The size of the TX_PATH FIFO:
Name: `m_fifo_size`
Type: `long long`

ii. The underflow Alarm level for the FIFO
Name: `UFL_TXF`
Type: `unsigned long long`

iii. To set TX_PATH in master mode

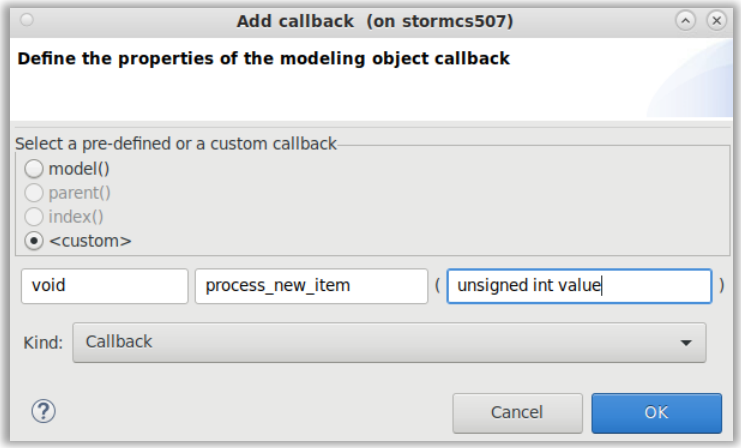
Name: `m_isDriver`
Type: `bool`

iv. Empty/full indicator flags
Name: `TX_FULL` and `TX_EMPTY`
Type: `bool`

v. The clock period for TX_PATH
Name: `clk_period`
Type: `sc_core::sc_time`

10. In the ***Callbacks*** sub-tab for the `TX_PATH` object, use the ***Add new callback*** icon ( ), or double-click open the ***Add callback*** dialog.

    i. Add a `process_new_item` callback that returns `void` and has an `unsigned int value` parameter as argument:



Note:

- Callbacks are to be called from within the reuse object (or hierarchy) but will be implemented in the model or parent hierarchy. They are to be used when the reuse object needs to trigger behavior in the model.

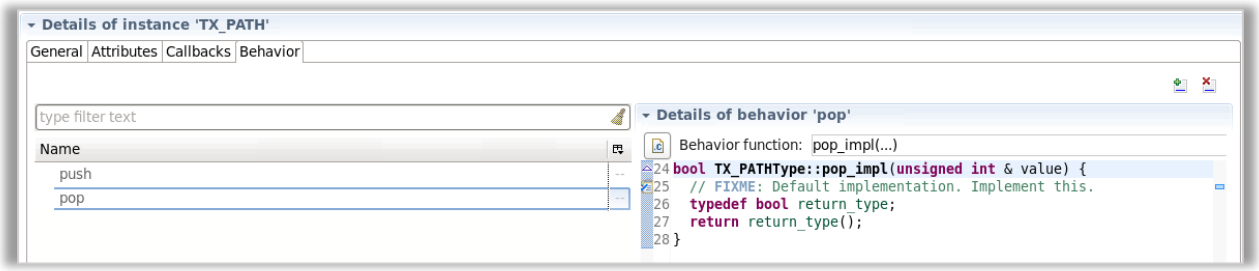
- There are pre-defined callbacks to get a pointer to the model, the parent hierarchy, or the index. Ideally these are to be avoided to enable better reuse of code but they are made available to avoid an overload of attributes and behaviors.

11. In the same way, in the ***Behavior*** sub-tab, add the behaviors for the `TX_PATH` object as follows.

    i. Add a `push` behavior returning `void` using a single argument to be set as `unsigned int value`.

    ii. Add a `pop` behavior returning a `bool` value using a single referenced argument to be set as `unsigned int & value`.

iii. Save and build the model – after that the default implementation for callbacks and behaviors are generated.



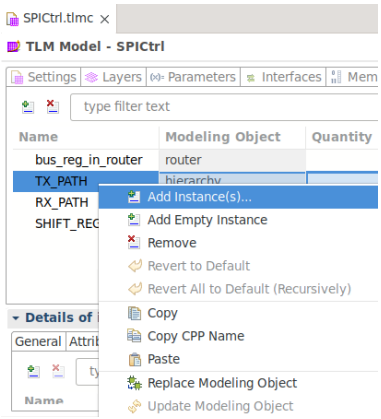
Note-: Behaviors are to be called from the model or parent hierarchy but are implemented in the reuse object. They are to be used to trigger behavior in the reuse object.

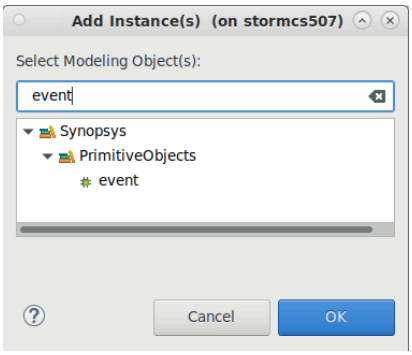
Task-5: The TX_PATH Internals

- ***Motivation**: The transmission logic is similar to a fifo, data can be pushed and popped from an internal fifo. It needs extra processing to be able to forward data to the SHIFT REGISTER and to notify the SPI Protocol engine that new data is available.*

12. For this, we add the following objects to the `TX_PATH` object:

    i. with ***Name*** `newItemEvent` of ***Type*** `event`. No further configuration is needed for this object.





    ii. with ***Name*** `TXFIFO` of ***Type*** `fifo`.

    iii. with ***Name*** `TXControl` of ***Type*** `process`.

Name	Modeling Object
TX_PATH	hierarchy
newItemEvent	event
TXFIFO	fifo
TXControl	process
RX_PATH	hierarchy
SHIFT_REGISTER	variable

Note: Check the ***Context Help*** on these reuse objects to get familiar with their features.

13. To configure the `TXFIFO` object, focus on the ***Attributes*** sub-tab, and add the following settings:

    iv. Set the value of the *size* attribute to `m_fifo_size`.

    v. Set the value of the *almost_empty_level* attribute to `UFL_TXF`, another attribute on the `TX_PATH` hierarchy reuse object. This is the underflow level

    vi. Set the value of the *almost_full_level* attribute to `m_fifo_size-1`

14. In the ***Callbacks*** sub-tab for the `TXFIFO` object, enable the following callbacks by setting the value in the ***Enabled*** column to `true`.

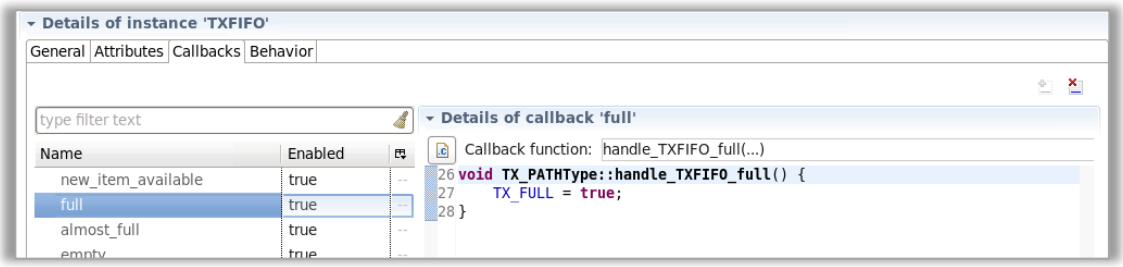
    i. ***Save*** the design to ensure the default implementations for the callbacks are generated.

    ii. Implement the `new_item_available` callback to notify the `newItemEvent` when the `TX_PATH` is in master mode. This will be used to initiate transactions on the serial interface. Add the following code into the peephole view that contains the implementation of the callback:

```
void TX_PATHType::handle_TXFIFO_new_item_available() {
        if (m_isDriver) {
                newItemEvent.notify(clk_period);
        }
}
```

    iii. Implement the `full` and `almost_full` callbacks to set the *TX_FULL* attribute on the hierarchy to true. Again, use the peephole view in the callbacks sub-tab to add the implementation of the callback:

```
void TX_PATHType::handle_TXFIFO_full() {
        TX_FULL = true;
}
```




```
void TX_PATHType::handle_TXFIFO_almost_full() {
        TX_FULL = true;
}
```

    iv. Implement the `empty` and `almost_empty` callbacks to set *TX_EMPTY* attribute on the hierarchy to true:

```
void TX_PATHType::handle_TXFIFO_empty() {
        TX_EMPTY = true;
}
```

```
void TX_PATHType::handle_TXFIFO_almost_empty() {
        TX_EMPTY = true;
}
```

v. The *TX_FULL* and *TX_EMPTY* attributes will later be used to drive interrupt control registers in the memory map.

15. Configure the `TxControl` process instance in the ***Attributes*** sub-tab by adding the following settings:

    i. For the *type* attribute, set the ***Value*** to `THREAD` - this will force the implementation of the process to be a SystemC thread, allowing for `wait()` calls and blocking interface calls.

    ii. Leave the *sensitivity {}[]* struct array unmodified - here you could define sensitivities that depend on other reuse object attributes or registers and bitfields.

    iii. Expand the *advanced_options* struct: set the ***Quantity*** of the *systemc_events[]* to `1` and specify `&newItemEvent` as value for *systemc_events[0]*. This will add the `newItemEvent` event to the sensitivity list of the process.

Name	Type	Quantity	Value	Kind	Description	Assignment	
type	process_type	---	THREAD	Const	---	---	--
▸ sensitivity{}[]	---	0	---	AttributeStruct	---	---	--
▾ advanced_options{}	---	---	---	AttributeStruct	---	---	--
stack_size	unsigned int	---		Const	---	---	--
▾ systemc_events[]	sc_core::sc_event*	1		Const	---	---	--
systemc_events[0]	---	---	&newItemEvent	---	---	---	--

Note: A pointer to the event needs to be specified in order to avoid that the event is copied (which is anyway not allowed).

16. In the ***Callbacks*** tab, specify the implementation of the mandatory `main` callback. This callback is the implementation of the process, so in case of a `THREAD` it is executed in a `while(1)` statement and preceded with a `wait` for the sensitivity.

    i. In the `main` callback, we will retrieve items from the fifo and forward them to the `process_new_item()` callback on the hierarchy. So, here we trigger the callback on the hierarchy, which will be implemented in the model.

```
void TX_PATHType::handle_TXControl_main () {
        while(!TXFIFO.is_empty()) {
                unsigned int lastExtractedTxFifoItem;
                TXFIFO.pop(lastExtractedTxFifoItem);
                process_new_item(lastExtractedTxFifoItem);
        }
}
```

Note: Putting the processing of new items in a separate thread allows to *fill the FIFO asynchronously from processing it*. This means that between signaling the event that a new element has arrived and finalizing the processing of that in the `process_new_item` callback, more elements may have been pushed in the FIFO, therefore we continue to process elements until it is empty.

17. Finally, we can now implement the behaviors of the `TX_PATH` hierarchy itself:

Select the `TX_PATH` hierarchy and go to the ***Behavior*** sub-tab. We have defined 2 behaviors for the `TX_PATH` object before: `push` and `pop`. We need to provide an implementation for each of those.

i. A default implementation for the behaviors should be generated earlier when saving the design. If it is not available, use the ***Generate Source Only*** option from the build button of the toolbar to make sure the default implementation is available.

For the `push` behavior:

```
void TX_PATHType::push_impl(unsigned int value) {
    TXFIFO.push(value);
    if (TXFIFO.nr_elements > TXFIFO.almost_empty_level)
        TX_EMPTY = false;
}
```

For the `pop` behavior:

```
bool TX_PATHType::pop_impl(unsigned int & value) {
    bool success = TXFIFO.pop(value);
    if (TXFIFO.nr_elements < TXFIFO.almost_full_level)
        TX_FULL = false;
    return success;
}
```

ii. Save and build the model.

Note: With this, we have *completed* the implementation of the TX_PATH hierarchical reuse object.

- It is constructed so that it encapsulates all the transmit behavior in a single reuse object with a set of configuration attributes, a callback and 2 behaviors.
- The implementation of this hierarchical reuse object is available in separate `.h` and `.cpp` files. So, it is possible to take this code and turn it into an actual reuse object. This is outside the scope of this training.
- It is important to structure code as much as possible as reusable components so that later if, and when, needed they can be easily converted into proper reuse objects.

Task-6: Using The TX_PATH instance

- ***Motivation****: The `TX_PATH` hierarchical reuse object looks as in the picture below. Next, we should integrate it into the model.*

18. First, we configure the `TX_PATH` instance by defining the values for its attributes (in the ***Attributes*** tab):

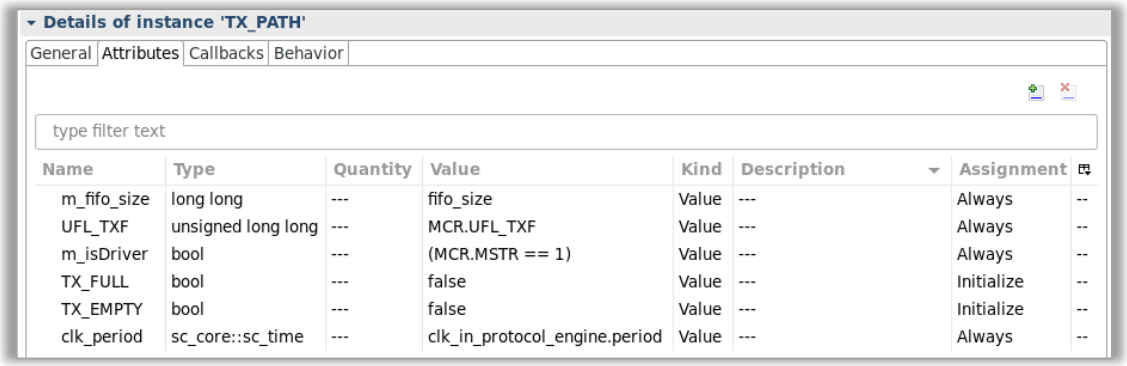
i. Set the ***Value*** of the *m_fifo_size* attribute to `fifo_size`, the size parameter for the model.

ii. Set the ***Value*** of the *UFL_TXF* attribute to `MCR.UFL_TXF`. This is a continuous assignment that will ensure that the value of the `UFL_TXF` bitfield in the register file of the *SPICtrl* model will be copied into the *UFL_TXF* attribute of the `TX_PATH` object.

iii. Set the ***Value*** of the *`m_isDriver`* attribute to `(MCR.MSTR == 1)` **-** another continuous assignment that keeps attribute and bitfield value in sync.

iv. Set the ***Value*** of the *TX_FULL* and *TX_EMPTY* attributes to `false` and set the ***Assignment*** column to `Initialize` (this is not a strict requirement since there is no dependency defined so the continuous assignment would never be triggered).

v. Set the ***Value*** of the *clk_period* attribute to `clk_in_protocol_engine.period`.

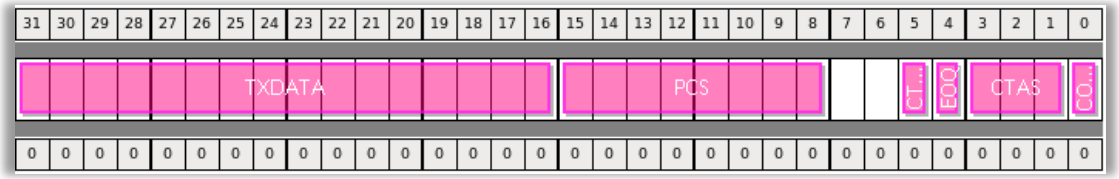


Name	Type	Quantity	Value	Kind	Description	Assignment	
m_fifo_size	long long	---	fifo_size	Value	---	Always	--
UFL_TXF	unsigned long long	---	MCR.UFL_TXF	Value	---	Always	--
m_isDriver	bool	---	(MCR.MSTR == 1)	Value	---	Always	--
TX_FULL	bool	---	false	Value	---	Initialize	--
TX_EMPTY	bool	---	false	Value	---	Initialize	--
clk_period	sc_core::sc_time	---	clk_in_protocol_engine.period	Value	---	Always	--

19. Next, we can implement the `process_new_item` callback of the `TX_PATH` object in the model.

i. The value passed in the `process_new_item` is the value in the `TXFIFO`, which will get pushed there through the *PUSHR* register. So, the structure of that data is as defined in the *PUSHR* register.

PUSHR	0x1c	---	Push register
CONT	---	0:0	Continuous selection flag
CTAS	---	1:3	CTAS register selection
EOQ	---	4:4	End of Queue
CTCNT	---	5:5	Clear Transfer Counter
PCS	---	8:15	Peripheral Chip Select
TXDATA	---	16:31	Data to transmit



ii. The data coming from the *PUSHR* register now needs to be processed to be able to start a transfer on the *Serial_IO* interface. To start a transaction, we need to know

a. Which *transfer configuration* which will be used. The transfer configurations are the *CTAR[]* registers. The *PUSHR.CTAS* bitfield selects an element from this array. From the transfer configuration, we need the Frame size (*FMSZ*) and the clock prescaling (*PCSSCK*) as follows.

CTAR[]	0x4	4	---	Transfer Configuration
FMSZ	---	---	0:4	Frame size in bits
CPOL	---	---	5:5	Clock Polarity
CPHIA	---	---	6:6	Clock Phase configuration
LSBFE	---	---	7:7	MSB or LSB first transmit
PCSSCK	---	---	8:9	Clock prescaler

b. From the *PUSHR* register data, we also need to extract:

- The peripheral chip select (*PCS*)
- The data to transmit (*TXDATA*)

c. With this data the *Serial_IO* interface can be programmed to initiate the transaction.

iii. Next the transfer complete bit needs to be set (*SR.TCF*)

iv. The transmit data should be copied to the *SHIFT_REGISTER*

v. This leads to the following code for the `process_new_item` callback:

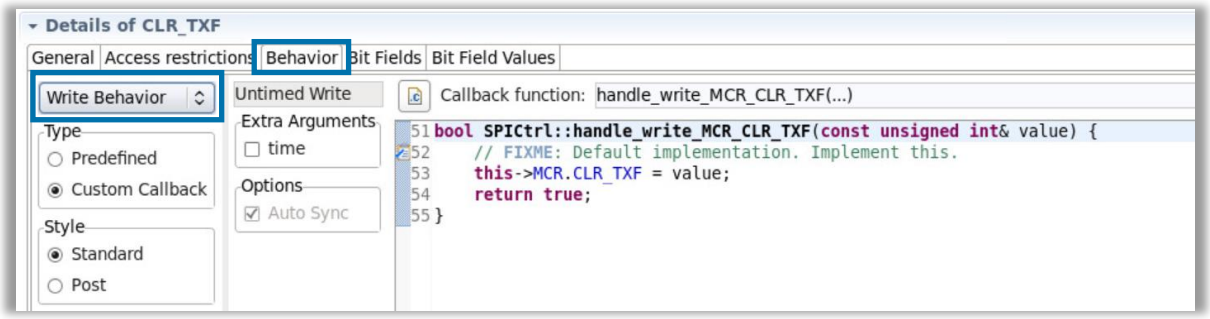
```
void SPICtrl::handle_TX_PATH_process_new_item (unsigned int value) {
    unsigned int iCtar = (value >> 1) & 0x7; // extract transfer configuration index
    unsigned int PCS = (value >> 8) & 0xFF; // get peripheral chip select
    unsigned int TXDATA = (value >> 16) & 0xFFFF; // get transfer data
    unsigned int pre = this->CTAR[iCtar].PCSSCK; // get clock prescaler for this configuration
    unsigned int nBits = this->CTAR[iCtar].FMSZ + 1; // get framesize for this configuration
    // start transfer on Serial_IO
    this->Serial_IO_protocol_engine.bit_time = this->spi_clk_protocol_engine.period / pre;
    this->Serial_IO_protocol_engine.start_master_transfer(TXDATA, nBits, PCS);

    this->SR.TCF = 1; // set transfer complete bit
    this->SHIFT_REGISTER = TXDATA; // copy transmit data to shift register
}
```

Task-7: Linking TX_PATH Behavior to the Memory Map Elements

- ***Motivation**: Now we can integrate the TX_PATH object with the register map:*
  - Enable to clear the fifo via the CLR_TXF bitfield in the MCR register
  - Integrate the TX_PATH object with the interrupt handling of the SPICtrl model
  - Feed the TX_PATH object via the PUSHR register

20. The *CLR_TXF* bitfield in the *MCR* register is to be used to clear the `TXFIFO`.

    i. This is done by adding an `Untimed Write` behavior to the *CLR_TXF* bitfield. Select the `Untimed Write` behavior from the dropdown menu in the ***Memory Map*** table (***Write Behavior*** column of *CLR_TXF* bitfield). Next, go to the ***Behavior*** sub-tab to edit the implementation of the callback. Make sure that also in the Behavior sub-tab the Write Behavior view is selected – otherwise the view will be empty:



    ii. Swap the default implementation of the callback for the following:

```
bool SPICtrl::handle_write_MCR_CLR_TXF(const unsigned int& value) {
        this->MCR.CLR_TXF = value;
        if (value == 1) this->TX_PATH.TXFIFO.clear();
        return true;
}
```

Note: Here we are accessing the `clear()` behavior of the `TXFIFO` object from outside the `TX_PATH` reuse object hierarchy. This is OK as it is a public API, but for cleaner encapsulation that API could be exposed on the boundary of the hierarchy.

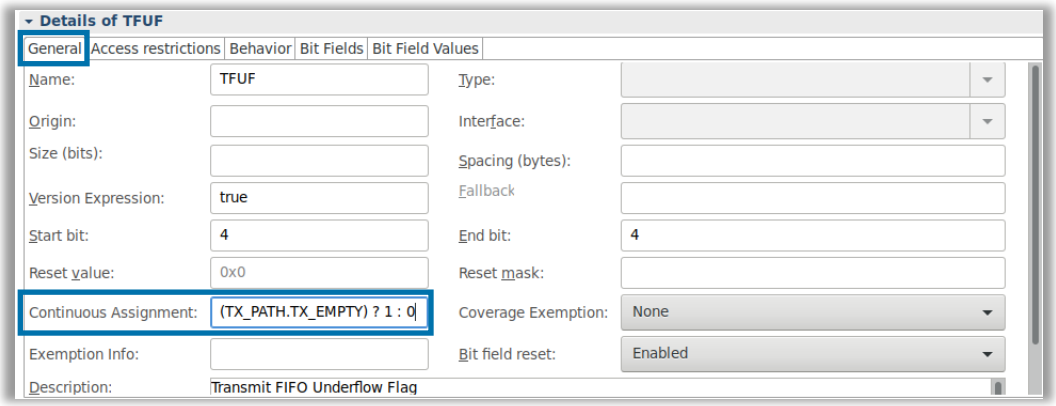
    iii. The *CLR_TXF* bitfield is also specified to always return 0 when read. This is done by adding an `Untimed Read (No Store)` callback. This is a callback that doesn't store the return value of the read into the register – swap the default implementation of the callback for the following:

```
bool SPICtrl::handle_read_MCR_CLR_TXF(unsigned int& value) {
        value =0;
        return true;
}
```

21. The *TFUF* and *TFFF* bitfields in the *SR* register indicate whether an underflow happens in the `TX_PATH` object or whether it is full. These bitfields can be defined through a ***Continuous Assignment*** (to be entered in the ***Details of TFUF/TFFF > General*** tab) using the *TX_EMPTY* and *TX_FULL* attributes on the `TX_PATH` object as follows:

    i. *TFUF*: `(TX_PATH.TX_EMPTY) ? 1 : 0`

    ii. *TFFF*: `(TX_PATH.TX_FULL) ? 1 : 0`



    This will ensure that these bitfields toggle with the values of *TX_EMPTY* and *TX_FULL.*

22. In the previous lab (Module 2 Lab A), we already made sure that the respective interrupt signals (*tfuf_irq* and *tfff_irq*) are driven by continuous assignments using the according bitfields *TFUF* and *TFFF*. Finally, we will go to the *PUSHR* register. The *PUSHR* register will implement a standard `Write` callback, where the data is pushed into the `TXFIFO` object when the SPI controller is configured as master. This is shown below:

```
bool SPICtrl::handle_write_PUSHR(const unsigned int& value,
        const unsigned int& byteEnables, sc_core::sc_time& time) {
    this->PUSHR = ((this->PUSHR & ~byteEnables) | (value & byteEnables));
    this->TX_PATH.push( (unsigned int) this->PUSHR);
    return true;
}
```

23. However, this pushing data into the FIFO should only be allowed when the controller is enabled. This is controlled by the *MDIS* bitfield in the *MCR* register.

    i. Basically, when this bitfield has value `1`, the module is not operational. If the controller is in master mode, no new data is allowed on the `TXFIFO` object and in slave mode incoming transfers are ignored.

    ii. To model this for the transmit part, we will add an ***Access Restriction*** to the *PUSHR* register that takes the *MCR.MDIS* value into account. Select `Custom Simple Write` in the ***Access Restriction*** column for the *PUSHR* register. Add the following implementation in the Access restrictions subtab:

```
scml2::access_restriction_result SPICtrl::restrict_PUSHR (
            unsigned int& data, unsigned int& bit_enables) {
  if (this->MCR.MDIS == 1) {
      SCML2_WARNING(PROGRAMMING_WARNING) <<
           "Pushing data to the TX FIFO while controller is disabled" << endl;
      scml2::restrict_all(data, bit_enables);
  }
  return scml2::RESTRICT_NO_ERROR;
}
```

Note: There are FastTrack messages now with *severity* **warning** and *category* **PROGRAMMING_WARNING** to alert a software developer of the wrong configuration of the peripheral.

24. Save and build the project

Task-8: Implementing the RX_PATH Object (reception control)

- ***Motivation**: Now we will implement the functionality for the RX_PATH. Just like the transmission control the reception control is similar to a fifo. The RX_PATH is a minimal wrapper around a fifo object, adding a couple of extra flags to the fifo.*

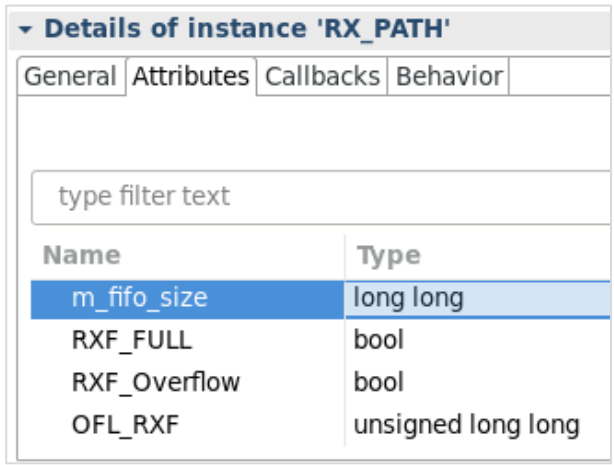
25. Just like before we start by setting up the interface of the *RX_PATH* object hierarchy by adding its attributes, callbacks, and behaviors via the details panel in the ***Instances*** tab. In the **Attributes** tab for the hierarchical reuse object *RX_PATH*, add the following elements:

    i. The size of the RX_PATH FIFO:
    Name: `m_fifo_size`
    Type: `long long`

    ii. The overflow Alarm level for the FIFO
    Name: `OFL_RXF`
    Type: `unsigned long long`

    iii. RXFifo full indicator flag
    Name: `RXF_FULL`
    Type: `bool`

    iv. Fifo overflow indicator flag
    Name: `RXF_Overflow`
    Type: `bool`



26. In the ***Behavior*** tab,

    i. Add a *push* behavior that returns `void` and has a single `unsigned int` argument `value`.

    ii. Add a *pop* behavior that returns a `bool` and has a single `unsigned int` reference argument `value` (i.e. `unsigned int & value`)

27. The `RX_PATH` object only contains a fifo, so select the `RX_PATH` hierarchical instance in the instances tab and use <CTRL-n> to add an instance to the hierarchy. Specify it with ***Name*** `RXFIFO` and ***Modeling Object*** type `fifo`.

RX_PATH	hierarchy
RXFIFO	fifo

28. Specify the values for the attributes of the `RXFIFO` instance in its ***Attributes*** tab:

    i. Set the value for *size* to `m_fifo_size` (the attribute from `RX_PATH`)

    ii. Set the *almost_empty_level* to `1`.

    iii. Set the *almost_full_level* to `OFL_RXF` (the attribute from `RX_PATH`)

Details of instance 'RXFIFO'

General | Attributes | Callbacks | Behavior

type filter text

Name	Type	Quantity	Value
size	long long	---	m_fifo_size
almost_empty_level	unsigned long long	---	1
almost_full_level	unsigned long long	---	OFL_RXF
nr_elements	unsigned long long	---	---

29. Only the `full` and `almost_full` callbacks for the `RXFIFO` need to be implemented in the ***Callbacks*** tab: (make sure to first enable them by setting the ***Enabled*** column to `true`)

    i. Both callbacks are used to set the corresponding flag on the `RX_PATH` hierarchy:

```
void RX_PATHType::handle_RXFIFO_full () {
        RXF_FULL = true;
}
```

```
void RX_PATHType::handle_RXFIFO_almost_full () {
        RXF_Overflow = true;
}
```

Note:

- There is no need to monitor that the receive fifo is empty, the purpose of the fifo-full/empty flags is to allow the SW to throttle the data it is writing to the SPICtrl or to ensure that the SW catches up with the incoming data.

- There is an option to use the `new_item_available` callback but as the new data will be handled in the SPI protocol engine already, we don't need it here.

30. Next, we can add the implementation of the *push* and *pop* behaviors on the `RX_PATH`. Select the `RX_PATH` object and go to the ***Behavior*** sub-tab in the details panel.

    i. The *pop* implementation is used to remove an element from the fifo and to update the *RXF_FULL* and *RXF_Overflow* flags:

```
bool RX_PATHType::pop_impl(unsigned int & value ) {
    bool result = RXFIFO.pop(value);
    RXF_FULL = false;
    if (RXFIFO.nr_elements < RXFIFO.almost_full_level)
        RXF_Overflow = false;
    return result;
}
```

    ii. The push implementation simply pushes data into the fifo:

```
void RX_PATHType::push_impl(unsigned int value )  {
    RXFIFO.push(value);
}
```

This completes the definition of the `RX_PATH` object.

31. Save and build the project.

Task-9: Using the RX_PATH Instance

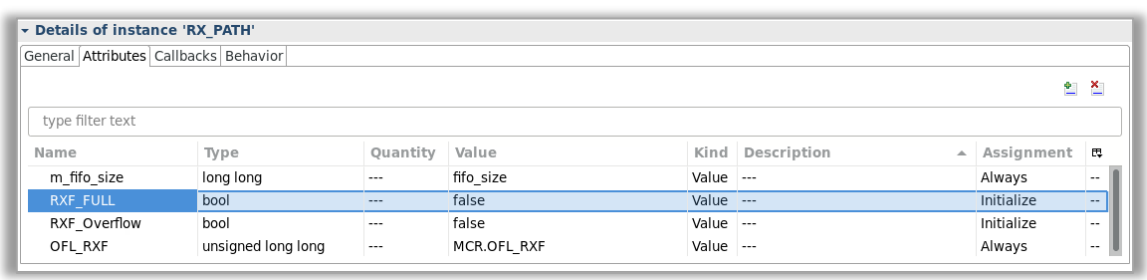
- ***Motivation****: With the RX_PATH defined we can now integrate the instance into the model.*

32. First we define the values for the `RX_PATH` attributes in its ***Attributes*** sub-tab:

    i. Set the ***Value*** of the *m_fifo_size* attribute to `fifo_size` (the size parameter for the model).

    ii. Set the ***Value*** of the *OFL_RXF* attribute to `MCR.OFL_RXF`. This is a continuous assignment that will ensure that the value of the *OFL_RXF* bitfield in the register file of the SPICtrl model gets copied into the *OFL_RXF* attribute of the `RX_PATH` object.

    iii. Set the ***Value*** of the *RXF_FULL* and *RXF_Overflow* attributes to `false` and set the ***Assignment*** column to `Initialize`.
    (This is not a strict requirement since there is no dependency defined so the continuous assignment would never be triggered.)



33. Second, we need to link the register behavior to the `RX_PATH` object

i. Go to the *MCR* register and select the *CLR_RXF* bitfield. Reading this bitfield should always return 0. Writing 1 to this bitfields will clear the content of the FIFO. Hence, we will implement the following callbacks:

   a. ***Read Behavior***: `Untimed Read (No Store)`

   b. ***Write Behavior***: `Untimed Write`

Name	Origin	Size/Quantity	Bit Range	Access Restrictions	Read Behavior	Write Behavior	Description
m_regs	0x0	0x24 bytes	---				simple top level register bank that is bound to the
MCR	0x0		---				Main Control Register
MSTR	---	---	0:0				Master/Slave mode configuration bit
PCSSE	---	---	6:6				Enable Peripheral Chip select in Strobe mode
MDIS	---	---	17:17				Disable device
CLR_TXF	---	---	20:20		Untimed Read (No Store)	Untimed Write	clear/flush TX FIFO
CLR_RXF	---	---	21:21		Untimed Read (No Store)	Untimed Write	clear/flush RX FIFO

ii. The implementation of these callbacks can be added directly in the Peephole view:

```
bool SPICtrl::handle_read_MCR_CLR_RXF (unsigned int& value) {
        value =0;
        return true;
}
```

```
bool SPICtrl::handle_write_MCR_CLR_RXF (const unsigned int& value) {
        this->MCR.CLR_RXF = value;
        if (value == 1) this->RX_PATH.RXFIFO.clear();
        return true;
}
```

iii. In the *POPR* register, implement a `Read (No Store)` callback to read the data from the *RXFIFO*. Any write access should be ignored and therefore the ***Access Restriction*** is set to `Read Only Ignore`. The `Read (No Store)` read behavior callback implementation is shown below.

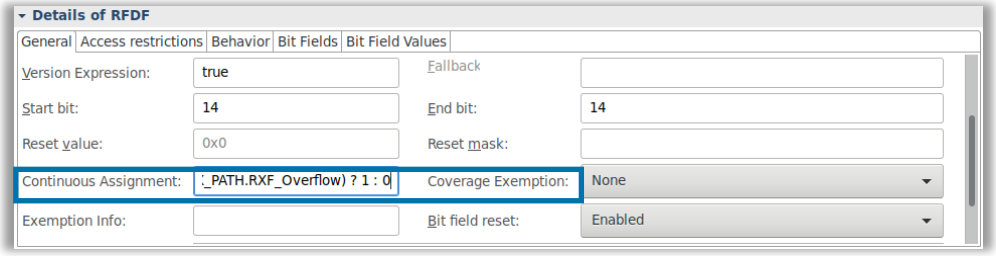
```
bool SPICtrl::handle_read_POPR(unsigned int &value,
            const unsigned int &byteEnables, sc_core::sc_time& time) {
    unsigned int tempData = 0;
    this->RX_PATH.pop(tempData);
    this->POPR = tempData;
    value = ((this->POPR & byteEnables) | (value & ~byteEnables));
    return true;
}
```

It pops a value from the `RX_PATH`, stores it in the *POPR* register and then makes sure that the return value is according to the `byte_enable` settings of the transaction request.

34. The *RFOF* and *RFDF* bitfields in the *SR* register indicate whether an overflow happens in the `RX_PATH` or whether it is drained. These bitfields can be defined through a ***Continuous Assignment*** *(in the* ***Details > General*** *sub-tab)* using the *RXF_Overflow* and *RXF_FULL* attributes on the `RX_PATH` as follows:

   i. *RFOF*: `(RX_PATH.RXF_FULL) ? 1 : 0`

   ii. *RFDF*: `(RX_PATH.RXF_Overflow) ? 1 : 0`



This will ensure that these bitfields toggle with the values of TX_EMPTY and TX_FULL.In the previous lab (Module 2 Lab A), we already made sure that the respective interrupt signals (*rfof_irq* and *rfdf_irq*) are driven by continuous assignments using the according bitfields *RFOF* and *RFDF*.

This completes the integration of the `RX_PATH` object into the model.

35. Save and build the project.

Task-10: Configuring the SPI Protocol Engine

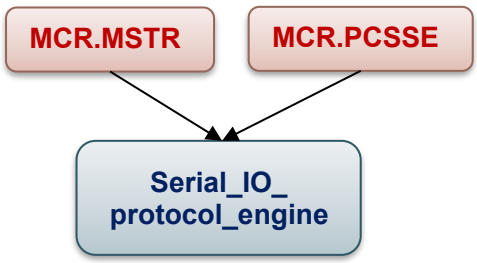
- ***Motivation**: The SPI protocol engine is automatically inserted when an SPI interface instance is added to the model. The configurable attributes (`bit_time` and `check_clocks`) were introduced and configured in the previous lab (Module 2 Lab A). Next, we will describe what other programmable options exist to configure the SPI protocol engine during execution.*

Besides the ***Attributes*** part, the SPI protocol engine has other methods to configure its functionality. This includes the following aspects:

i. Configuring it as *Initiator* or *target* on the SPI interface.

ii. Configuring the selection of slaves as *Chip Enable* or *Strobe* type.

iii. Configuring the *bit_time* for a master transfer.



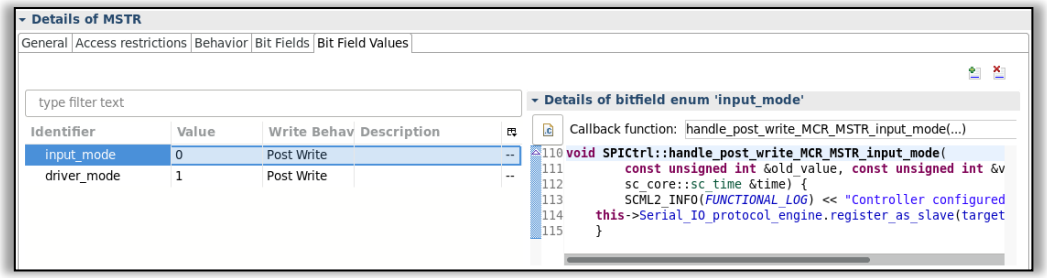
36. To control the *Initiator* or *target* configuration of the SPI interface, the *MSTR* bitfield of the *MCR* register will be used. Click on the *MSTR* bitfield, expand the ***Details of MSTR*** panel and go to the ***Bit Field Values*** sub-tab. Add the following two `Enum` values:

    i. ***Identifier***: `input_mode`
    Value: `0`
    Write Behavior: `Post Write`

    This will generate a `handle_post_write_MCR_MSTR_input_mode` callback.

    ii. ***Identifier***: `driver_mode`
    Value: `1`
    Write Behavior: `Post Write`

    This will generate a `handle_post_write_MCR_MSTR_driver_mode` callback.



The implementation for both callbacks are shown below. The *target_id* is a parameter configured by the integrator of the model.

Note how we have added a FastTrack message with severity `info` and category `FUNCTIONAL_LOG` to inform a software developer about the new configuration of the peripheral.

```
void SPICtrl::handle_post_write_MCR_MSTR_input_mode(
const unsigned int& old_value, const unsigned int& value, sc_core::sc_time& time) {
    SCML2_INFO(FUNCTIONAL_LOG) << "Controller configured as Target." << std::endl;
    this->Serial_IO_protocol_engine.register_as_slave(target_id);
}
```

```
void SPICtrl::handle_post_write_MCR_MSTR_driver_mode(
const unsigned int& old_value, const unsigned int& value, sc_core::sc_time& time) {
    SCML2_INFO(FUNCTIONAL_LOG) << "Controller configured as Initiator." << std::endl;
    this->Serial_IO_protocol_engine.register_as_master();
}
```

37. Similarly, the *chip enable* or *strobe* configuration of the SPI interface is done through the *PCSSE* bitfield of the *MCR* register. Go to the *PCSSE* bitfield, expand its ***Details*** tab and go to the ***Bit Field Values*** sub-tab. Add the following two `Enum` values:

    i. ***Identifier***: `chip_select_mode`
       Value: `0`
       Write Behavior: `Post Write`

    ii. ***Identifier***: `strobe_mode`
       Value: `1`
       Write Behavior: `Post Write`

The implementation for the above callbacks is shown below – they configure the chip addressing of the SPI protocol engine.

```
void SPICtrl::handle_post_write_MCR_PCSSE_chip_select_mode(
const unsigned int& old_value, const unsigned int& value, sc_core::sc_time& time) {

    this->Serial_IO_protocol_engine.set_slave_select_type_chip_select();
}
```

```
void SPICtrl::handle_post_write_MCR_PCSSE_strobe_mode(
const unsigned int& old_value, const unsigned int& value, sc_core::sc_time& time) {

    this-> Serial_IO_protocol_engine.set_slave_select_type_strobe_select();
}
```

Finally, we also need to add the behavior of the model when data is received on the *Serial_IO* interface. This is done by implementing the `start_received_transfer` and `end_received_transfer` callbacks on the protocol engine.

38. Go to the ***Interfaces*** tab, select the *Serial_IO* interface, and in the ***Callbacks*** sub-tab of the ***Details*** panel, add an implementation for both callbacks as follows.

    i. When a transfer is coming in we need to move the incoming data to the *SHIFT_REGISTER* and in case it the model is set in slave mode, we should start a slave transfer with the data we pop from the `TX_PATH` object. Change the default implementation for the `start_received_transfer_64bit_V3` callback as follows.

```
void SPICtrl::handle_Serial_IO_protocol_engine_start_received_transfer_64bit_V3 (
        unsigned data, unsigned bit_length) {
    this->SHIFT_REGISTER = data;
    if (this->Serial_IO_protocol_engine.getIsSlave()) {
        unsigned int value;
        if (TX_PATH.pop(value) == false) {
        SCML2_ERROR(BUFFER_UNDERFLOW) <<
                        "Transmit Fifo is empty, transferring unknown data" << endl;
        }
        Serial_IO_protocol_engine.start_slave_transfer(value, bit_length);
    }
}
```

ii. At the end of the transfer, we should push the data we stored in the *SHIFT_REGISTER* into the `RX_PATH` object. Change the default implementation for the `end_received_transfer` callback as follows.

```
void SPICtrl::handle_Serial_IO_protocol_engine_end_received_transfer () {
    RX_PATH.push(SHIFT_REGISTER);
}
```

39. Save and build the project.

We have now finished the implementation of the SPI Controller in its basic version.

What's Next

In the next lab, we will add versioning support to the SPI Controller model and implement additional functionality that is only available in the newer version of the SPI Controller. Versioning control allows you to maintain multiple versions of the same model in one TLMC project.